

WanChan Beam Breed-Classifer Fine-Tuning

Model development, controlled ablations, and validation results

Technical report | WanChan Beam | 21 August 2026

Summary. This report describes the development of the breed-classification stage in WanChan Beam. Starting with an ImageNet-pretrained MobileNetV3-Large, the study compared classifier heads, fine-tuning depth, optimization settings, input resolution, and augmentation choices on the official Tsinghua Dogs validation split. The final model uses 256 x 256 whole-dog crops, unfreezes the final feature block, keeps BatchNorm running statistics fixed, and applies Random Erasing during training. In a separate CPU float32 evaluation, it reached 85.04% top-1 and 93.83% top-2 accuracy across 5,200 images. Because the same validation split was used for model development, these figures describe validation performance rather than performance on an untouched test set.

1. Methods

1.1 Objective and study design

Breed recognition is the second stage of the WanChan Beam inference pipeline. A detector first locates each dog in an image, and the classifier then produces a probability distribution over 130 breed labels for every crop. MobileNetV3-Large was chosen because it was designed for efficient inference on mobile hardware while retaining strong image-classification performance [2]. Top-1 accuracy served as the main selection criterion, with validation cross-entropy loss and top-2 accuracy used as supporting measures.

Rather than testing every possible combination of settings, experiments were organized as a sequence of controlled ablations. At each stage, one main choice was varied and the preferred setting was carried into the next stage. This kept the development process manageable and made the effect of each decision easier to follow. All comparisons used the official validation split, and the main ablation table reports the best-accuracy checkpoint from each experiment.

1.2 Dataset, split, and label space

Training used the low-resolution release of the Tsinghua Dogs dataset [1]. The dataset contains 70,428 images from 130 breeds, with one dog per image and separate annotations for the whole body and head. Its official split provides 65,228 training images and 5,200 validation images, corresponding to 40 validation examples per breed. Checks of the supplied split files found no overlap between training and validation entries and no references to missing images or annotations.

The class vocabulary was derived directly from the training split. Breed names were read from the parent directory in each train.lst entry, deduplicated, sorted with Python's default string ordering, and assigned indices from 0 to 129. The resulting ordering was stored with every checkpoint and was identical across all 98 retained checkpoints.

1.3 Crop construction and preprocessing

Whole-body annotations were used so that training crops matched the output expected from the production detector. Bounding-box coordinates were parsed as integers and clamped to the decoded RGB image dimensions before cropping. The right and bottom coordinates therefore followed PIL's exclusive convention. Of 1,216 boxes that extended beyond an image boundary, 1,215 became valid after clamping. The remaining box was degenerate, so the existing fallback used the full RGB image. No sample was discarded, and no padding was added before the crop was taken.

Table 1. Selected training transform pipeline.

Order	Operation	Configuration
1	Horizontal flip	p = 0.5
2	Affine transform	RandomApply p = 0.5; rotation +/-8 degrees; translate = (0.04, 0.04); scale = (0.95, 1.05); shear = None; bilinear; fill RGB (124, 116, 104)
3	Color jitter	brightness = 0.15; contrast = 0.15; saturation = 0.10; hue = 0.02
4	Resize	Preserve aspect ratio; longest dimension -> 256; rounded output dimensions; PIL bilinear resampling
5	Square padding	Center on 256 x 256 RGB canvas filled with (124, 116, 104); odd extra pixel remains right/bottom
6	Tensor conversion	to_tensor(); RGB values scaled from 8-bit integers to float [0, 1]

Order	Operation	Configuration
7	Normalization	mean = [0.485, 0.456, 0.406]; std = [0.229, 0.224, 0.225]
8	Random Erasing	After normalization; p = 0.25; scale = (0.02, 0.15); ratio = (0.5, 2.0); value = 0; inplace = False

Validation preprocessing used the same crop, aspect-preserving resize, mean-RGB padding, tensor conversion, and ImageNet normalization, but omitted all stochastic augmentation. The earlier frozen-backbone head comparison used 224 x 224 inputs with horizontal flipping and ColorJitter; affine augmentation and Random Erasing were not used at that stage.

1.4 Architecture and fine-tuning strategy

The classifier was based on torchvision's mobilenet_v3_large initialized with MobileNet_V3_Large_Weights.DEFAULT. In torchvision 0.28.0, this setting resolves to the IMAGENET1K_V2 weights. The selected head kept the pretrained 960-to-1,280 linear projection and Hardswish activation, followed by Dropout(p = 0.4) and a new Linear(1,280, 130) output layer. Two simpler alternatives, Linear(960, 130) and 960-to-512-to-130, were evaluated during the initial head comparison. Bias terms were retained in all linear layers.

For the initial head comparison, the pretrained 960-to-1,280 projection remained frozen and only the final 1,280-to-130 layer was trained. Subsequent experiments unfroze the full classifier together with zero, one, two, or three of the deepest feature modules. The selected model included model.features[16] and the classifier, for a total of 1,552,130 trainable parameters. BatchNorm2d layers stayed in evaluation mode so that their running statistics were fixed. Within features[16], however, the BatchNorm affine weights and biases remained trainable and were optimized with the feature parameter group.

1.5 Optimization, sampling, and checkpointing

Table 2. Training protocols for the baseline and selected fine-tuning stage.

Component	Initial head training	Selected fine-tuning
Epoch budget	10	5
Batch size	64	64
Optimizer	AdamW	AdamW; separate classifier and feature groups [3]
Learning rate	1e-3	Classifier 5e-5; features[16] 5e-6
Weight decay	1e-4	0.01 for all trainable tensors
Scheduler	None	CosineAnnealingLR; T_max = 5; eta_min = 0; step once per epoch
Training loss	Cross-entropy	Cross-entropy; label smoothing = 0.02
Validation loss	Cross-entropy	Cross-entropy; no label smoothing
Early stopping	Not used	Patience = 2; reset if either loss or accuracy strictly improves
Checkpointing	Legacy single best	Best loss, best accuracy, and latest after every epoch

Class imbalance was handled with a WeightedRandomSampler. Each image in class c was assigned weight $1/N_c$, and 65,228 samples were drawn with replacement in each epoch using a dedicated CPU generator. Both data loaders used four workers; CUDA runs also enabled pinned memory and persistent workers. Incomplete batches were retained. MixUp (alpha = 0.2) and CutMix (alpha = 1.0) were tested as separate batch-level augmentations and, when enabled, were applied to every training batch [4,5]. Neither was used in the final configuration.

GPU training used CUDA autocast with GradScaler, while the CPU fallback used standard float32 computation. For the selected run, the PyTorch, CUDA, and sampler seeds were set to 43. Python's and NumPy's random-number generators were not seeded, and deterministic algorithms, cuDNN determinism, and custom worker initialization were not enabled. The run is therefore reproducible at the level of its recorded setup, but exact bit-for-bit repetition is not guaranteed.

1.6 Evaluation protocol

Validation was performed with `model.eval()` inside `torch.inference_mode()`. Mean cross-entropy was calculated for each batch, multiplied by the batch size, and then summed before division by the 5,200 evaluated images. This produced a sample-weighted loss and included the final 16-image batch correctly. Top-1 and top-2 predictions were obtained with `topk(k = 2, dim = 1)` and accumulated per image. The final verification run processed the complete validation split on CPU in float32 using the deterministic validation pipeline.

1.7 Reproducibility and implementation details

Record	Value
Software and hardware	Python 3.13.6; PyTorch 2.13.0+cu130; Torchvision 0.28.0+cu130; CUDA runtime 13.0; NVIDIA RTX A2000 12 GB; Windows/WDDM
Source code	https://github.com/dinosoldic/wanchan-beam ; audited commit 0631db3f15864dfe2f97aaacea3ce2ac2c43c9f3; ML source tree clean when reviewed
Training-revision provenance	The checkpoint does not record a Git commit. The revision above identifies the implementation reviewed for this report, not a confirmed historical training revision.
Selected model artifact	Best-accuracy checkpoint from seed 43, epoch 4
Checkpoint SHA-256	534BF22CCD19DD45BF8F046D2C6922FCDF901B5AFF1D0DD27A4D3BF18A390630
Checkpoint size	30,184,747 bytes
Split SHA-256	train.lst: A225719140667BCB54E20E05D785700D33C36C3ADFF71310FF2AE3DEAA4DE6D6; validation.lst: 0CBC806C3D587B4947623C1F8FC8BF3E601EACD6879AC74CF08507FB4F2C5F42

2. Results

2.1 Frozen-backbone baseline and head comparison

Among the frozen-backbone models, the head that retained the 1,280-unit hidden layer performed best. Its loss-selected checkpoint reached a validation loss of 0.5406 and 82.73% top-1 accuracy at epoch 3; the preserved clean evaluation gave 82.75% top-1 and 93.08% top-2 accuracy. By comparison, the linear and 512-unit heads reached 82.58% and 82.04% top-1, respectively. The 1,280-unit design was therefore used in the fine-tuning experiments.

2.2 Sequential ablation results

Table 3. Settings carried forward through the sequential model-selection study. *Metrics are from each stage's best-accuracy checkpoint and are rounded for presentation.*

Stage	Evaluated values	Selected	Epoch	Val. loss	Top-1
Head	Linear; 512; 1,280 hidden	1,280	3	0.5406	82.73%
Fine-tuning depth	Classifier only; deepest 1, 2, 3	features[16]	3	0.5489	83.06%
Classifier LR	1e-4; 5e-5	5e-5	4	0.5389	83.31%
Feature LR	1e-5; 5e-6	5e-6	3	0.5345	83.25%
LR schedule	Fixed; cosine	Cosine	2	0.5326	83.21%
Label smoothing	0.02; 0.05; 0.10*	0.02	3	0.5314	83.48%
Affine	Disabled; enabled	Enabled	5	0.5275	83.63%
Weight decay	0; 0.01; 0.1	0.01	5	0.5275	83.67%
Dropout	0; 0.4; 0.5	0.4	5	0.5252	83.75%
Input size	224; 256	256	4	0.4927	84.88%
BatchNorm	Frozen stats; adaptation	Frozen	4	0.4927	84.88%
Batch mixing	None; MixUp 0.2; CutMix 1.0	None	4	0.4927	84.88%

Stage	Evaluated values	Selected	Epoch	Val. loss	Top-1
Random Erasing	$p = 0; 0.25; 0.50$	$p = 0.25$	4	0.4907	84.92%
Seed	42; 43; 44	43	4	0.4872	85.08%

No otherwise-identical run with zero label smoothing was retained. The value 0.02 was selected from the smoothing values that were explicitly evaluated, so this result should not be read as evidence that label smoothing itself outperformed a matched zero-smoothing control.

Some settings were chosen by considering accuracy and loss together rather than by taking the highest top-1 value alone. For example, label smoothing of 0.05 reached 83.69% top-1 but raised the loss to 0.5692, whereas 0.02 gave a lower loss with 83.48% top-1. A weight decay of 0.1 improved top-1 over 0.01 by only 0.02 percentage points and also produced a slightly higher loss, so 0.01 was retained. The clearest single improvement came from increasing the input resolution from 224 to 256 pixels, which raised top-1 accuracy from 83.75% to 84.88%.

Neither MixUp nor CutMix improved on the 256-pixel control. Best-checkpoint top-1 accuracy was 84.81% for MixUp with label smoothing 0.02, 84.77% for MixUp without label smoothing, and 84.56% for CutMix. Random Erasing with $p = 0.25$ reached 84.92%. Increasing the probability to 0.50 lowered top-1 to 84.79%, despite a slightly lower checkpoint loss, so $p = 0.25$ was selected on the primary metric [6].

2.3 Stability across random seeds

Table 4. Matched Random Erasing comparison across seeds. Each row uses the best-accuracy checkpoint.

Seed	No erasing: top-1 / loss	$p = 0.25$: top-1 / loss	Change	Selected
42	84.88% / 0.4927	84.92% / 0.4907	+0.04 pp	$p = 0.25$
43	85.04% / 0.4888	85.08% / 0.4872	+0.04 pp	$p = 0.25$
44	84.85% / 0.4942	84.90% / 0.4932	+0.06 pp	$p = 0.25$
Mean	84.92% / 0.4919	84.97% / 0.4904	+0.04 pp	$p = 0.25$

The gain from Random Erasing was modest but consistent across the three tested seeds. With $p = 0.25$, top-1 accuracy increased and validation loss decreased in every matched comparison. The average improvement was about 0.04 percentage points, or roughly two additional correct predictions in a 5,200-image validation run.

2.4 Selected model and final validation

Table 5. Selected deployment-candidate configuration.

Component	Selected value
Backbone	MobileNetV3-Large; ImageNet IMAGENET1K_V2 weights
Classifier	960 -> 1,280 -> Hardswish -> Dropout(0.4) -> 130
Input	RGB whole-body crop; proportional resize and padding to 256 x 256
Trainable network	Complete classifier plus features[16]; 1,552,130 trainable parameters
BatchNorm	Running statistics frozen; affine parameters in features[16] trainable
Optimization	AdamW; classifier LR 5e-5; feature LR 5e-6; weight decay 0.01; cosine schedule
Regularization	Label smoothing 0.02; dropout 0.4; Random Erasing $p = 0.25$
Selected checkpoint	Seed 43 best-accuracy checkpoint; epoch 4

Table 6. Final validation results. *The clean report values are from CPU float32 evaluation unless stated otherwise.*

Metric	Result
Checkpoint validation loss	0.4872046981
Stored CUDA/AMP top-1	85.076923%
Clean CPU float32 top-1	85.04%
Clean CPU float32 top-2	93.83%
Correct top-1 predictions	4,422 / 5,200
Clean frozen-baseline top-1	82.75%
Improvement over baseline	+2.29 percentage points; approximately 119 additional correct predictions

The checkpoint recorded 85.08% top-1 accuracy during CUDA mixed-precision validation. The independent CPU float32 evaluation produced 85.04%, corresponding to two fewer correct predictions. The CPU result is reported as the headline value because it came from the final, standalone verification pass.

2.5 Class-level error profile

Performance varied considerably across breeds. The lowest per-class accuracies were observed for `chinese_rural_dog` and `toy_poodle` (27.5% each), followed by `papillon` (45.0%), `Pekinese` (50.0%), and `Black_sable` (52.5%). Common directed confusions included `Black_sable` to `German_shepherd`, `toy_poodle` to `teddy`, `teddy` to `toy_poodle`, `Australian_terrier` to `silky_terrier`, and `English_foxhound` to `Walker_hound`. Most of these errors occurred between visually similar classes or labels that are difficult to distinguish from appearance alone.

References

1. D.-N. Zou, S.-H. Zhang, T.-J. Mu, and M. Zhang, "A new dataset of dog breed images and a benchmark for finegrained classification," *Computational Visual Media*, vol. 6, pp. 477-487, 2020. doi: 10.1007/s41095-020-0184-6.
2. A. Howard et al., "Searching for MobileNetV3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 1314-1324, 2019. doi: 10.1109/ICCV.2019.00140.
3. I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019. <https://openreview.net/forum?id=Bkg6RiCqY7>
4. H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," in *International Conference on Learning Representations (ICLR)*, 2018. <https://openreview.net/forum?id=r1Ddp1-Rb>
5. S. Yun, D. Han, S. J. Oh, J. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization strategy to train strong classifiers with localizable features," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 6023-6032, 2019. doi: 10.1109/ICCV.2019.00612.
6. Z. Zhong, L. Zheng, G. Kang, S. Li, and Y. Yang, "Random erasing data augmentation," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 7, pp. 13001-13008, 2020. doi: 10.1609/aaai.v34i07.7000.